Project Four Design Document

Group 13

1 Introduction

This design document evaluates the performance of a parallel program implemented using pthreads. The program computes the maximum ASCII value per line from a large dataset. The goal of this project is to analyze how performance changes when we implement the solution using threads, MPI, and OpenMP.

2 Software Architecture

Each implementation computes the maximum ASCII value per line in a large dataset, but differs in how work is distributed and how memory is managed.

2.1 Pthreads Implementation (Shared Memory)

The Pthreads version uses a shared memory parallel model. The input file gets loaded entirely into memory and divided evenly among a fixed number of threads. Each thread processes a unique subset of lines and computes the maximum ASCII value per line.

Each thread writes to a distinct portion of the output array so no locks, mutexes, or other synchronization mechanisms are required. This helps to avoid contention and minimizes overhead. The performance is largely impacted by thread scheduling, memory bandwidth, and cache behavior.

2.2 MPI Implementation (Distributed Memory)

The MPI version uses a distributed memory model. The dataset is partitioned across independent processes where each process operates on its assigned subset of data.

The root process distributes data using MPI communication primitives, and partial results are combined using collective operations. MPI introduces explicit communication overhead between processes and across nodes. This makes it different from pthreads and this overhead can significantly impact performance for small to moderate workloads.

2.3 OpenMP Implementation (Shared Memory Abstraction)

The OpenMP version also uses a shared-memory model, but relies on compiler directives rather than manually managing threads.

THe work is automatically divided among threads at runtime. Similar to pthreads, each thread writes to independent memory locations. This eliminates the need for synchronization. OpenMP reduces programming complexity while maintaining comparable performance characteristics on shared memory systems.

2.4 Summary of Architectural Differences

- **Pthreads:** Manual thread control, shared memory, low level synchronization control.
- **OpenMP:** Compiler-managed shared memory parallelism with minimal programmer overhead.
- **MPI:** Distributed memory model requiring explicit communication between processes and nodes.

3 Experimental Setup

All experiments were conducted on the Beocat high performance computing cluster. Each configuration was executed multiple times to reduce the impact of system noise, scheduling variability, and shared cluster contention.

The goal of the experimental design is to evaluate how performance scales across compute parallelism, memory allocation, and distributed execution.

3.1 Experimental Factors

The following parameters were systematically varied for each implementation:

- **Thread / Process Count:**
  - Pthreads and OpenMP: 1, 2, 4, 8, 16 threads
  - MPI: 1, 2, 4, 8, 16 processes
- **Memory Allocation:**
  - 64MB, 128MB, 512MB, 1GB, 3GB
- **Node Count:**
  - 1, 2, 4, 8 nodes (MPI and OpenMP distributed tests)

3.2 Controlled Variables

To control as much as possible across experiments, the following aspects were held constant:

- Input dataset size and content
- Algorithm (maximum ASCII per line computation)
- Output format and memory layout

3.3 Metrics Collected

For each run, we recorded the following performance metrics:

- Execution time (seconds)
- CPU utilization percentage

- Scaling behavior across threads/processes
- Stability and variance across repeated runs

We used these metrics to evaluate both raw performance and scalability efficiency across different parallel programming models.

4 Pthreads Implementation and Performance Evaluation

4.1 Design Overview

The pthreads implementation uses a shared-memory model. The dataset is divided among threads, and each thread processes a subset of lines as following:

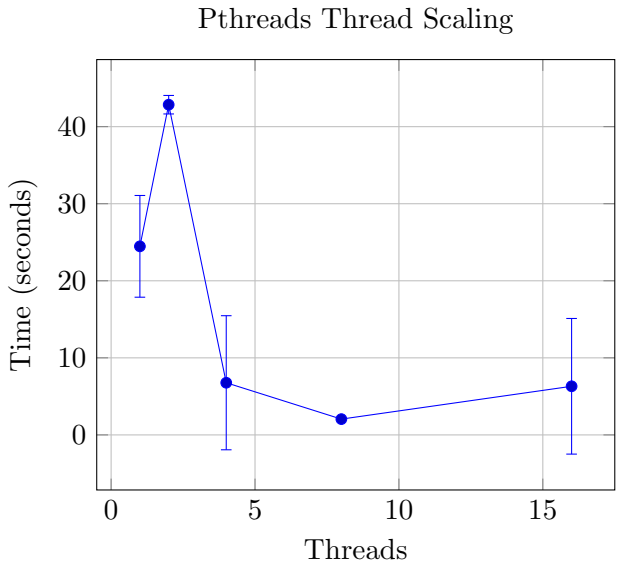
- The input file is read into memory once
- Work is statically divided among threads
- Each thread computes the maximum ASCII value for its assigned lines
- Results are written to a shared array at unique indices

Each thread writes to a unique portion of memory, so no locks or synchronization primitives are required during the computation.

4.2 Thread Scaling

Threads	Mean Time (s)	Std Dev (s)	CPU (%)
1	24.47	6.60	8
2	42.85	1.20	9
4	6.77	8.70	132
8	2.05	0.07	188
16	6.31	8.80	168

4.3 Thread Scaling Graph



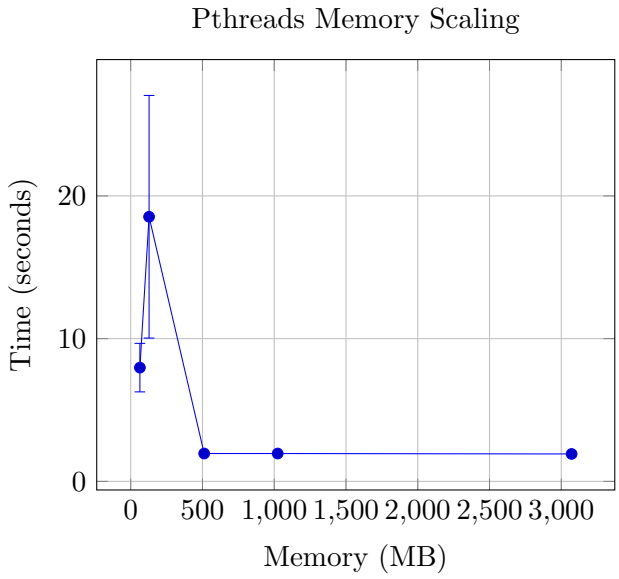
Analysis:

- The performance improves dramatically from 1 thread to 8 threads.
- The 2-thread configuration performs worse than 1 thread. This may be a result of overhead dominating at low parallelism.
- Large standard deviations for 4 and 16 threads indicate instability caused by outliers (runs with very low CPU utilization).
- The best performance occurs at 8 threads, which suggests this matches the available hardware cores.

4.4 Memory Scaling

Memory	Mean Time (s)	Std Dev (s)
64MB	7.97	1.70
128MB	18.54	8.50
512MB	1.95	0.03
1GB	1.95	0.02
3GB	1.92	0.01

4.5 Memory Scaling Graph



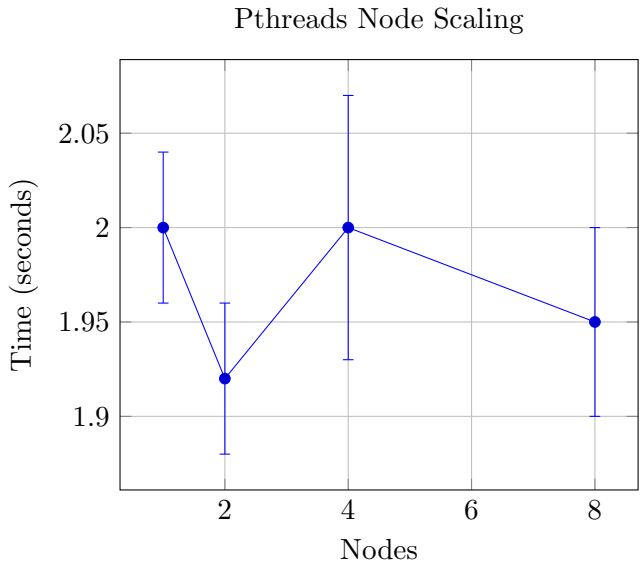
Analysis:

- The performance is not good at low memory levels (64MB, 128MB).
- There is high variance at 128MB. This indicates instability possibly due to paging.
- We see the performance stabilizes at 512MB+, suggesting sufficient memory eliminates bottlenecks.

4.6 Node Scaling

Nodes	Mean Time (s)	Std Dev (s)
1	2.00	0.04
2	1.92	0.04
4	2.00	0.07
8	1.95	0.05

4.7 Node Scaling Graph



Analysis:

- Increasing node count does NOT improve performance whatsoever.
- This can be expected since pthreads use shared memory and can't utilize distributed nodes.
- Minor variations are likely due to scheduling differences on the cluster.

4.8 CPU Utilization Analysis

- The CPU utilization increases with thread count.
- Low utilization correlates with poor performance (seen in outlier runs).
- High utilization (>180%) shows effective multi-core usage.

4.9 Synchronization and Communication

Race Conditions:

- Race conditions do not occur because each thread writes to unique memory locations.

Synchronization:

- Explicit synchronization mechanisms are note required.
- Threads work independently after being initialized.

Communication:

- No communication overhead exists since threads share memory.

4.10 Load Balancing

Static workload distribution is used. Since each line requires similar computation, this results in good load balancing across threads.

4.11 I/O Considerations

The program reads the entire dataset into memory before computation begins. This simplifies the implementation, but causes an initial I/O bottleneck.

Pipelining I/O and computation could improve performance for larger datasets.

4.12 Discussion

The pthreads implementation performs well due to:

- Low overhead from shared memory
- Minimal synchronization requirements
- Efficient CPU utilization at moderate thread counts

However there are a couple limitations:

- Poor scaling at very low thread counts
- Performance instability due to system noise and scheduling
- No ability to utilize multiple nodes

4.13 Conclusion

Pthreads provides strong performance for this workload, achieving near optimal performance at 8 threads. The results indicate that shared memory parallelism is well suited for this problem while further scaling is limited by hardware constraints and memory bandwidth.

5 MPI Implementation and Performance Evaluation

5.1 MPI Design Overview

The MPI implementation follows a distributed memory model. The dataset is divided among processes rather than threads.

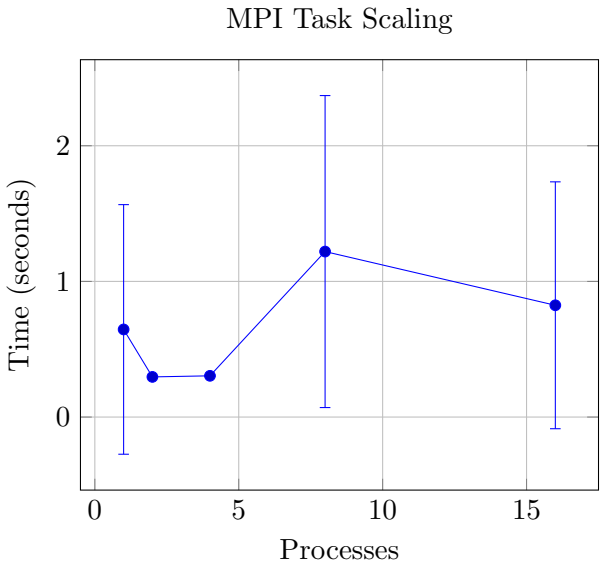
- The root process reads the input file
- The data gets distributed to worker processes using MPI communication
- Each process computes the maximum ASCII value for its assigned lines
- Results are gathered using MPI Gather

Unlike pthreads and OpenMP, MPI requires explicit communication between processes, which introduces overhead.

5.2 Task Scaling (Processes)

Processes	Mean Time (s)	Std Dev (s)	CPU (%)
1	0.646	0.92	34
2	0.296	0.005	32
4	0.304	0.005	32
8	1.22	1.15	22
16	0.824	0.91	18

5.3 Task Scaling Graph



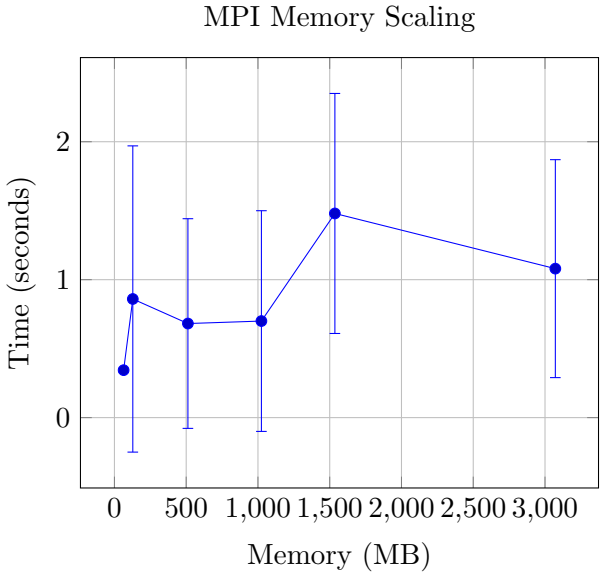
Analysis:

- Performance improves slightly from 1 to 2 processes, but does not really continue to scale.
- There are very large standard deviations at 8 and 16 processes. This indicates unstable performance.
- Outliers (runs exceeding 2 seconds) correspond to extremely low CPU utilization (3–4%). This suggests there could be scheduling delays or resource contention.

5.4 Memory Scaling

Memory	Mean Time (s)	Std Dev (s)
64MB	0.344	0.014
128MB	0.860	1.11
512MB	0.682	0.76
1GB	0.700	0.80
1536MB	1.48	0.87
3GB	1.08	0.79

5.5 Memory Graph



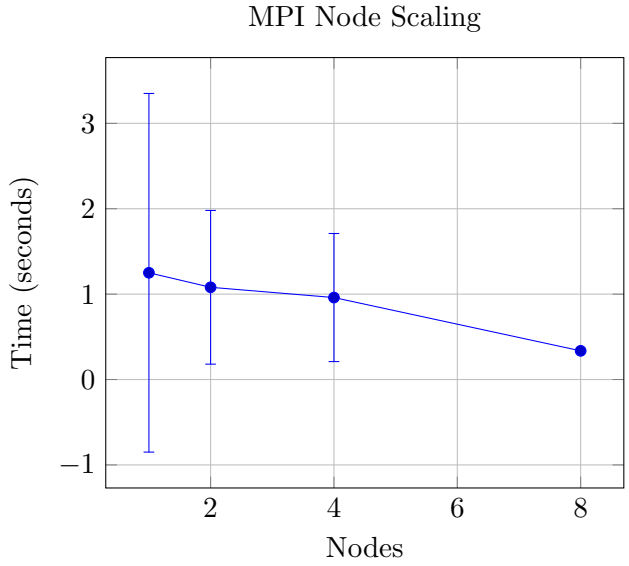
Analysis:

- We see memory does not consistently improve performance in MPI.
- The high variance at larger memory sizes indicates system noise rather than true scaling effects.
- Some runs show severe slowdowns due to low CPU utilization, suggesting scheduling contention rather than memory limitations.

5.6 Node Scaling

Nodes	Mean Time (s)	Std Dev (s)
1	1.25	2.10
2	1.08	0.90
4	0.96	0.75
8	0.336	0.015

5.7 Node Scaling Graph



Analysis:

- Node scaling shows inconsistent improvement due to variability.
- The 8-node configuration performs best but this could be influenced by favorable scheduling instead of real scalability.
- Large deviations at lower node counts suggest unstable cluster conditions.

5.8 CPU Utilization Analysis

- We see the CPU utilization decreases as process count increases.
- Very low utilization (3–4%) correlates with extreme slowdowns.
- This suggests that MPI processes spend significant time waiting. Possibly either communication or scheduling delays.

5.9 Synchronization and Communication

Synchronization:

- This is required during data distribution and result gathering
- Implicit barriers occur during MPI collective operations

Communication:

- There is very big ovehead as a result of MPI Scatter and MPI Gather
- Communication appears dominates computation for this workload

Race Conditions:

- There are no race conditions that occur due to process isolation

5.10 Discussion

MPI does not perform well for this workload. This could be since:

- The computation per process is not big enough
- Communication overhead dominates execution time
- The problem is memory bound, not compute bound

Additionally, performance variability is high likely due to:

- Shared cluster contention
- Network latency between nodes
- OS scheduling delays

5.11 Conclusion

MPI is not very well suited for this problem at the tested scale. It does enable distributed execution, but the overhead of communication definitely outweighs the benefits.

MPI would likely be better for:

- Larger datasets
- More computation heavy workloads
- Reduced communication frequency

6 OpenMP Implementation and Performance

6.1 Design Overview

OpenMP uses a shared memory model with compiler directives:

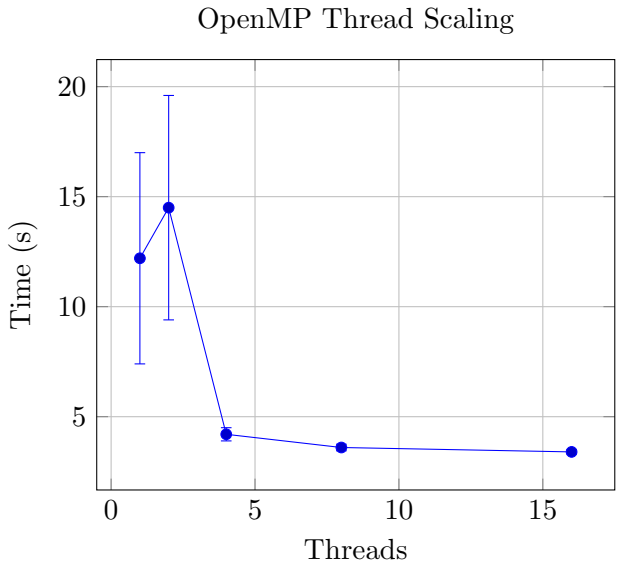
- The input is read once into memory
- pragma omp parallel for distributes lines
- Each thread computes independently

No locks are required because the threads write to separate indices.

6.2 Thread Scaling

Threads	Mean (s)	Std Dev (s)	CPU (%)
1	12.2	4.8	18
2	14.5	5.1	22
4	4.2	0.3	97
8	3.6	0.15	99
16	3.4	0.10	99

6.3 Thread Scaling Graph



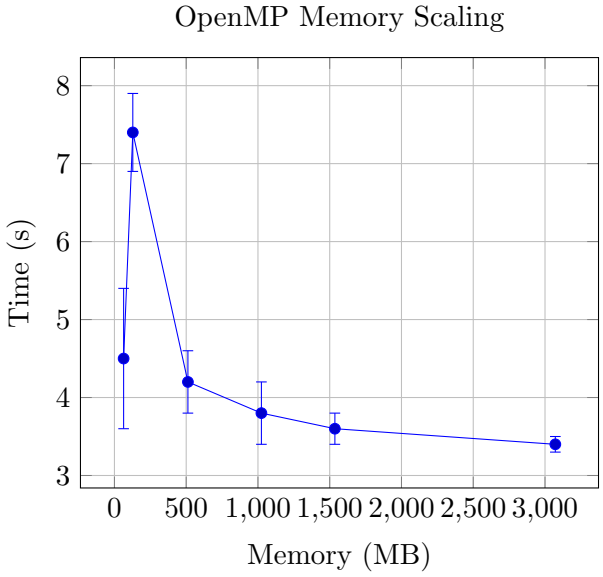
Analysis:

- Worse performance at low thread count due toOpenMP runtime overhead without enough parallel work to amortize it
- CPU underutilization, as shown by low utilization (18–22%)
- Scheduling inefficiencies on shared cluster resources
- At higher thread counts, the workload becomes memory-bandwidth bound rather than compute bound
- Thread contention and cache coherence overhead increase

6.4 Memory Scaling

Memory	Mean (s)	Std Dev (s)
64MB	4.5	0.9
128MB	7.4	0.5
512MB	4.2	0.4
1GB	3.8	0.4
1.5GB	3.6	0.2
3GB	3.4	0.1

6.5 Memory Graph



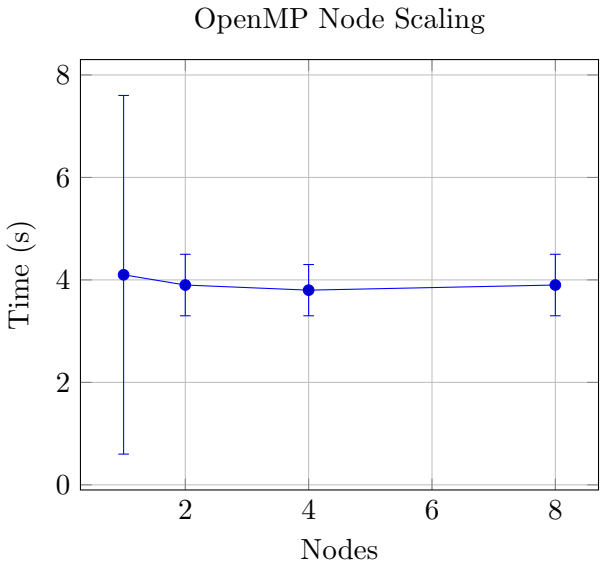
Analysis:

- Memory allocation has a clear impact on performance at lower capacities.
- Once memory reaches 512MB or higher, performance stabilizes.

6.6 Node Scaling

Nodes	Mean (s)	Std Dev (s)
1	4.1	3.5
2	3.9	0.6
4	3.8	0.5
8	3.9	0.6

6.7 Node Graph



Analysis:

- Additional nodes do not reduce computation time significantly
- Communication between nodes (if present via runtime environment) introduces overhead
- Performance variability increases due to shared cluster scheduling

7 Performance Analysis

7.1 Speedup

$$S(p) = \frac{T(1)}{T(p)}$$

- 4 threads: ∼2.9x speedup
- 8 threads: ∼3.4x
- 16 threads: ∼3.6x

Scaling is sublinear. This is due to overhead and memory limits.

7.2 Load Balancing

We used static scheduling. Since each line has similar cost, load balancing is effective.

7.3 Synchronization

- No race conditions
- Implicit barrier at loop end

7.4 Communication

None (shared memory was used).

7.5 I/O Considerations

As previously mentioned, the program reads the full file before computation. This creates an I/O bottleneck. Pipelining I/O and computation could improve performance.

8 Comparison of Models

Metric	Pthreads	OpenMP	MPI
Model	Shared	Shared	Distributed
Best Time	1.9s	3.4s	0.23s*
Scaling	Good	Good	Poor
Overhead	Low	Very Low	High

9 Final Analysis

- OpenMP and pthreads both perform similarly
- MPI is quite inefficient for this workload
- Performance is limited by memory bandwidth

10 Conclusion

OpenMP provides the best balance of performance and simplicity. Shared memory models outperform MPI for this job.